UNIVERSITI TENAGA NASIONAL

The Energy University

SEM 1 2024/2025
CSEB3213
DATA STRUCTURE & ALGORITHMS

LECTURER'S NAME:
MADAM NAZIFFA RAHA BINTI MD NASIR

PREPARED BY:

Pavala Rani Sunderan
SW01083718

1.0 Executive Summary

Luggage is one of the essentials travelers bring when traveling somewhere. However, it is no secret that the luggage management system at airport terminals bothers passengers, and traveling is only enjoyable when all the necessary processes run smoothly. The luggage management system offered by airports is inefficient and flawed. This inefficiency often leads to mishandled luggage, resulting in misplaced or missing items. This will disrupt travel plans and dissatisfaction among passengers, which highlights the need for more reliable and effective solutions ensuring smooth handling of luggage.

Current issues with airports luggage management system are as follows:

1. Inefficient storage and retrieval

Inefficient storage and retrieval of luggage are among the challenges passengers must face. These issues cause long delays and disrupt airport operations. When luggage is stored haphazardly, the time required to locate specific luggage increases. Additionally, during peak travel times, flights are often delayed due to improper luggage loading and unloading caused by poor luggage management systems. This frequently results in items being skipped or left behind, creating a bottleneck effect that overwhelms both manual and automated luggage systems.

2. Outdated technology

Most systems implemented nowadays are outdated in an effort to save costs. However, this creates more problems, as evident in issues like misrouting luggage during retrieval at airport terminals. The outdated system not only slows down operations but also limits the ability to provide passengers with accurate details and updates about their belongings. This automatically reduces confidence in the airport's ability to manage their luggage.

3. Luggage Misplacement

Missing luggage is one of the most crucial problems that often occurs. This is primarily caused by the lack of tracking mechanisms offered by the system. The situation worsens when everything is handled manually, leading to items being misplaced or lost. When this happens, it becomes difficult to locate the missing luggage since no data is stored in the system for workers to retrieve. A similar issue arises when there is an overload of uncollected luggage at the terminals, further slowing down the process of managing and retrieving luggage.

To increase the effectiveness of this system we have improved the system with more structured and well-organized data storage to ensure smoothness in handling luggage at airport terminals.

2.0 Objective

There are several objectives for creating a luggage management system at airports to ensure effective handling of passengers' luggage during storage and retrieval in common or uncollected zones.An organized system which has sorting and tracking helps that the luggage is stored perfectly and retrieved quickly.It directly helps the airport staffs are able manage luggage effectively during peak days such as public holidays.

Authorization security is crucial to prevent scenarios where luggage is accidentally taken or stolen. To address this, passengers are required to provide specific details about their luggage before boarding the airplane, including the owner’s name, color, and dimensions (length, width, and height in cm). The owner's name plays a key role in validating their identity and finding the rightful owner of uncollected luggage. This process ensures that only authorized individuals can retrieve their luggage.

Lastly, a well-organized retrieval process is ensured through carefully planned steps, including a ready queue, a common collection space, and designated uncollected zones. This organization helps to reduce stress and confusion, as the proper structure of ready queues, common collection spaces, and uncollected zones makes it easier for passengers to retrieve their luggage without hesitation. Passengers experience reduced waiting times which helps them stay calm and relaxed. This efficiency minimizes frustration and allows passengers to collect their luggage quickly.

3.0 Scope / Modules

For this project, we choose a combination of Stack (Linked list implementation), Queue, Vector, and Static Array due to their efficiency in handling various operations related to luggage storage and retrieval.

This system consists of several modules as follows:

1. Cargo Storage The luggage is stored in cargo using a stack (linked list implementation). Operations include:
    - **Adding:** Push luggage onto the stack for storage.
    - **Removing:** Pop luggage off the stack during retrieval.
    - **Updating:** Locate luggage by ID in the stack and update fields such as label, description, or destination.
2. Ready Queue Management The ready queue (in the luggage reclaim area) uses a queue structure to handle luggage. Operations include:
    - **Adding:** Enqueue luggage to the ready queue.
    - **Removing:** Dequeue luggage for customer collection.
    - **Updating:** Traverse the queue to find luggage by ID and update the reclaim area assignment or priority as needed.
3. Common Area Management The common area (in the luggage reclaim area) uses a static array for temporary storage. Operations include:
    - **Adding:** Insert luggage into the array.
    - **Removing:** Perform a sequential search to find luggage and remove it.
    - **Updating:** Locate luggage by ID in the array and update the status field (e.g., "Claimed," "Rechecking").
4. Uncollected Zone Management Uncollected luggage is stored using a vector for flexibility in resizing. Operations include:
    - **Adding:** Insert luggage into the vector.
    - **Removing:** Use a sequential search to locate and remove luggage by ID.
    - **Updating:** Search for luggage by ID in the vector and modify fields such as counter assignment, timestamp, or user-specific notes.
5. Sorting
    - **Quick Sort:** Used to find the uncollected zone's room with the greatest empty space.
        - Rearrange the rooms' priority using the upper method.
        - The first room after sorting will be inserted with luggage.
6. Searching
    - **Sequential Search:** Used for:
        - Collecting luggage in the common space at the luggage reclaim area.
        - Finding luggage in a particular uncollected zone's room.

How the solutions meet data requirements:

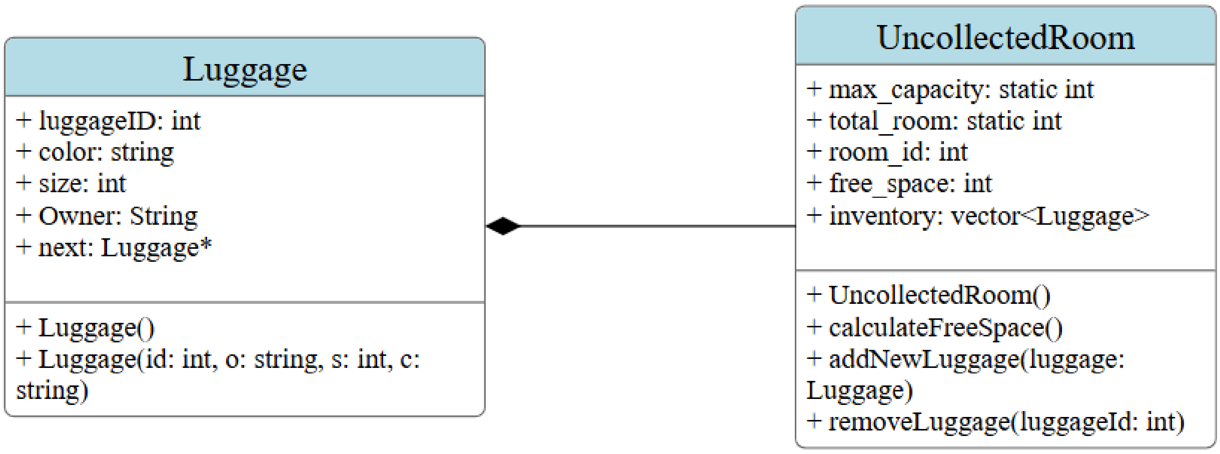
- **Efficient Storage and Retrieval:**
    - Queue operations ensure luggage is stored and retrieved in order, minimizing confusion.
    - Vector operations allow for quick location of uncollected luggage using a search algorithm.
- **Real-Time Tracking:**
    - Tracks luggage through its journey using queues and vectors, providing live updates.
    - Push operations move luggage to the ready queue, common area, or uncollected zone with clear categorization.
- **Handling Uncollected Luggage:**
    - Luggage automatically moves to the uncollected zone if unclaimed within a fixed time.
    - Dynamic resizing of vectors ensures no limit on storage capacity, adapting to varying luggage counts.

Proposed Analysis:

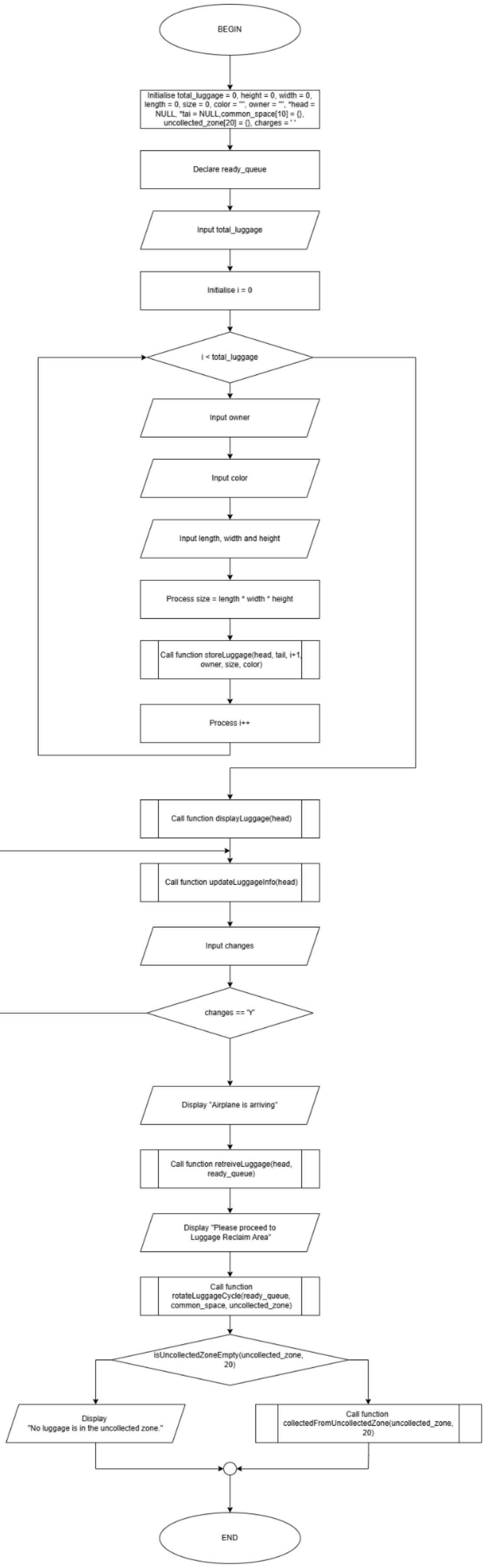
- **Time Complexity:**
    - Queue Operations:
        - Enqueue/Dequeue: O(1)
    - Vector Operations:
        - Linear Search: O(n)
        - Binary Search (sorted data): O(log n)
    - Sorting:
        - Quick Sort: O(n log n) for rearranging rooms in the uncollected zone.
- **Practical Improvements:**
    - Automating the movement of luggage reduces manual intervention, streamlining the process.
    - By halting operations when a user retrieves luggage, the system prevents errors and ensures user satisfaction.

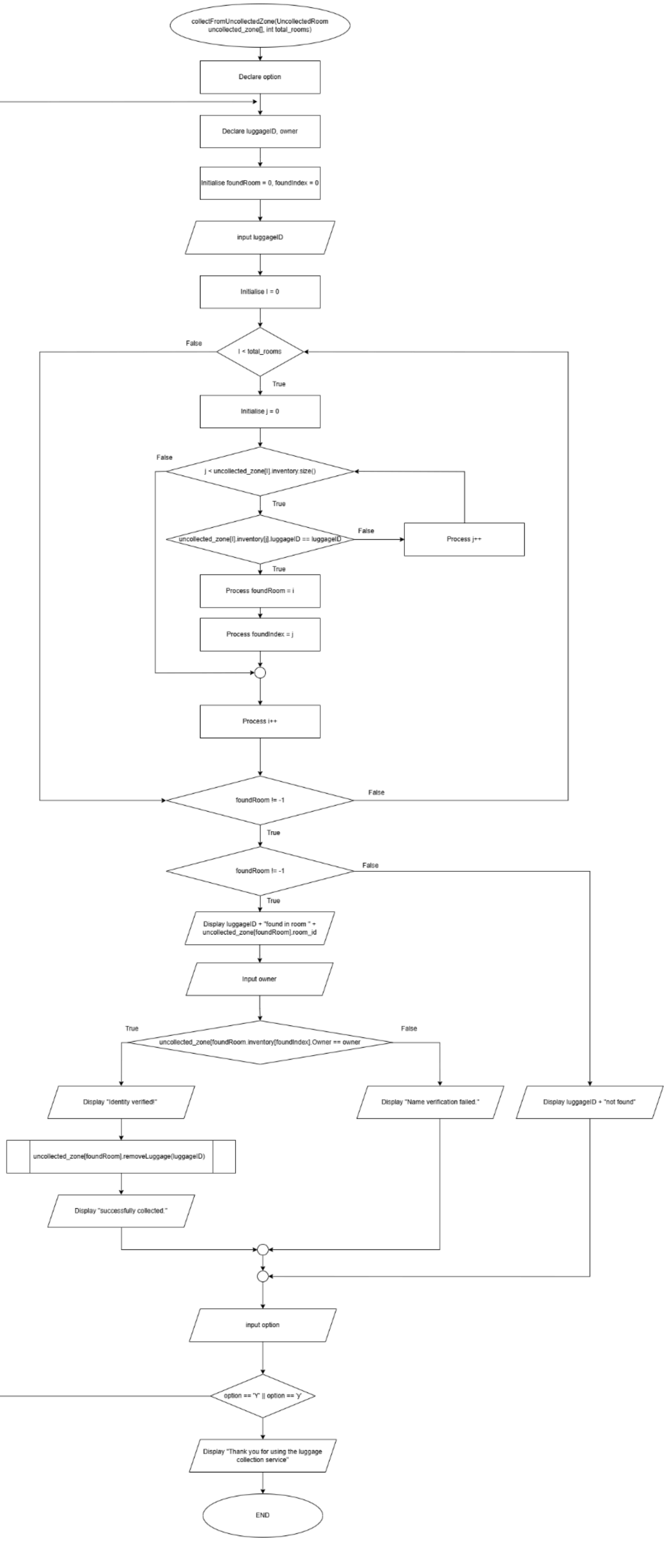
4.0 System Design

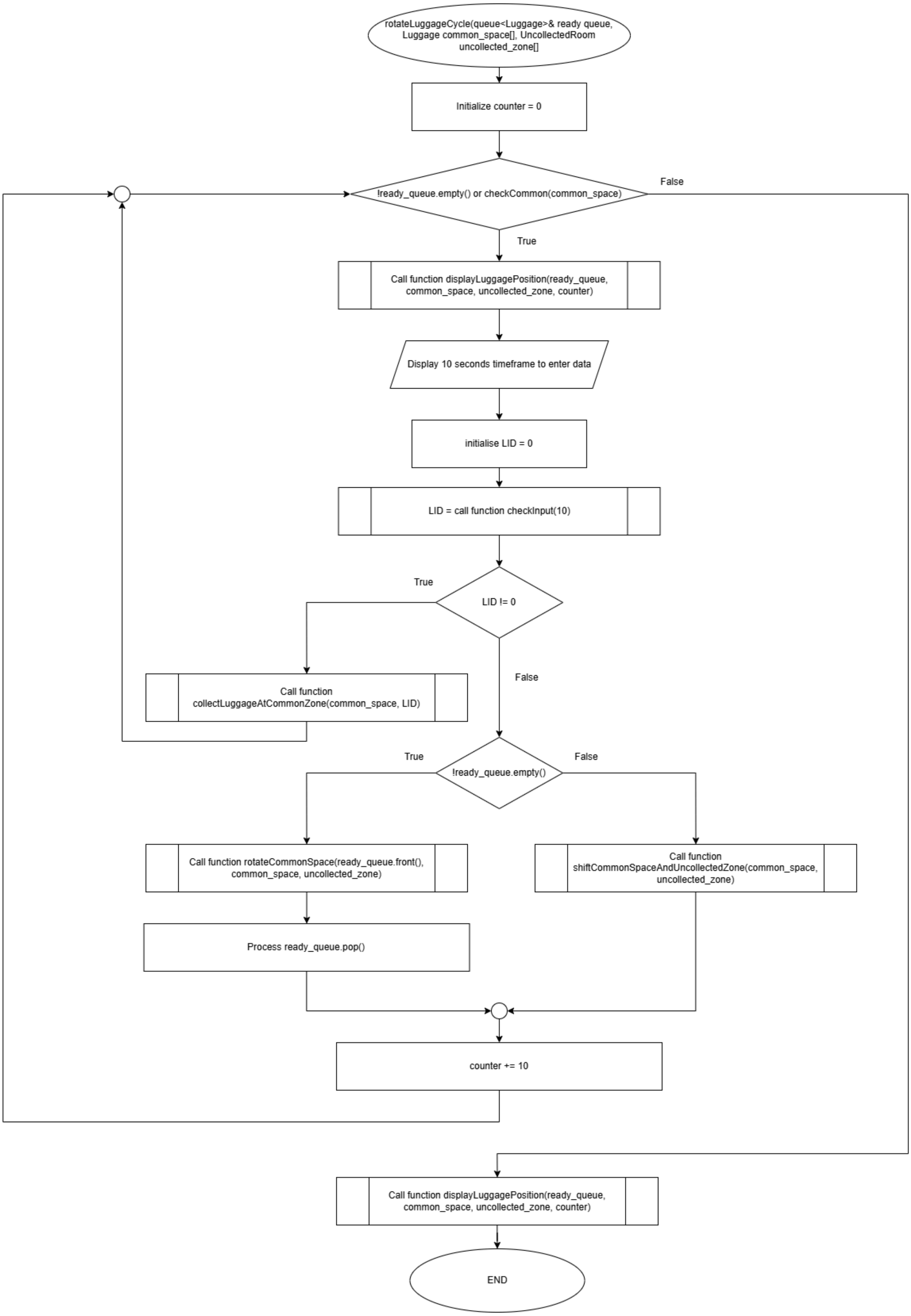
Class Diagram:



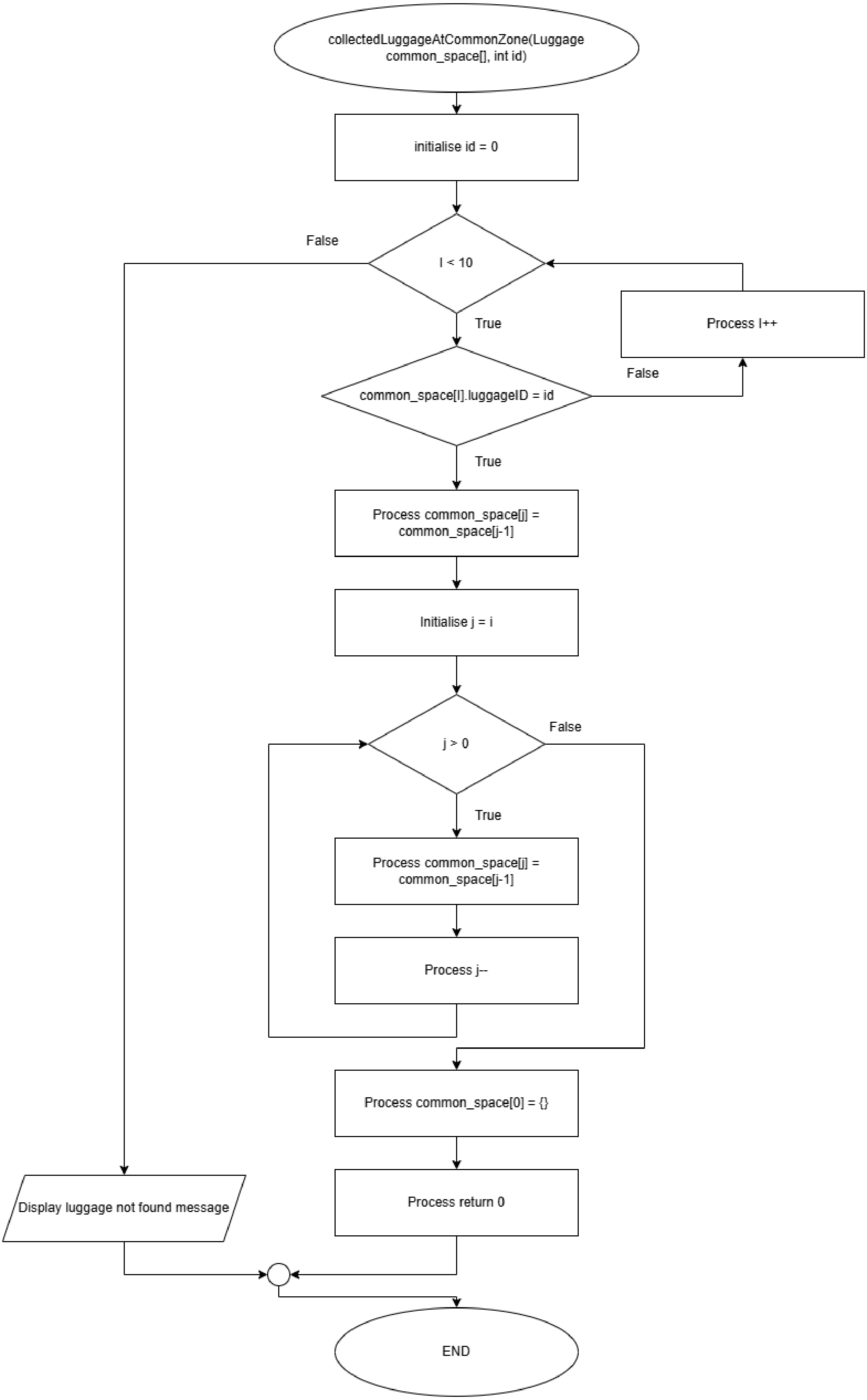
Flowchart:

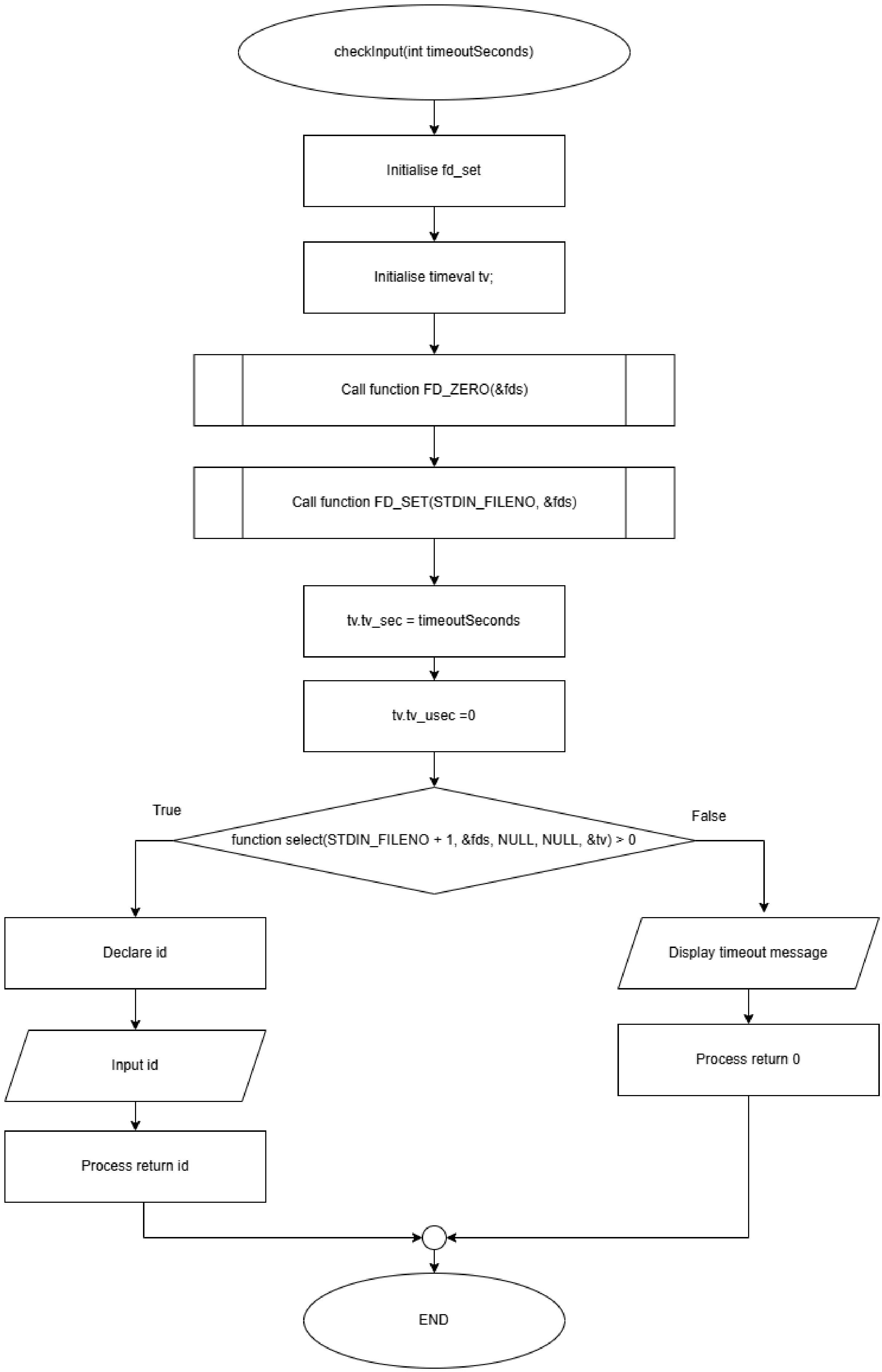

BEGIN
Initialise total_luggage = 0, height = 0, width = 0, length = 0, size = 0, color = "", owner = "", *head = NULL, *tai = NULL,common_space[10] = {}, uncollected_zone[20] = {}, charges = ' '
Declare ready_queue
Input total_luggage
Initialise i = 0
i < total_luggage
Input owner
Input color
Input length, width and height
Process size = length * width * height
Call function storeLuggage(head, tail, i+1, owner, size, color)
Process i++
Call function displayLuggage(head)
Call function updateLuggageInfo(head)
Input changes
changes == 'Y'
Display "Airplane is arriving"
Call function retreiveLuggage(head, ready_queue)
Display "Please proceed to Luggage Reclaim Area"
Call function rotateLuggageCycle(ready_queue, common_space, uncollected_zone)
isUncollectedZoneEmpty(uncollected_zone, 20)
Display "No luggage is in the uncollected zone."
Call function collectedFromUncollectedZone(uncollected_zone, 20)
END

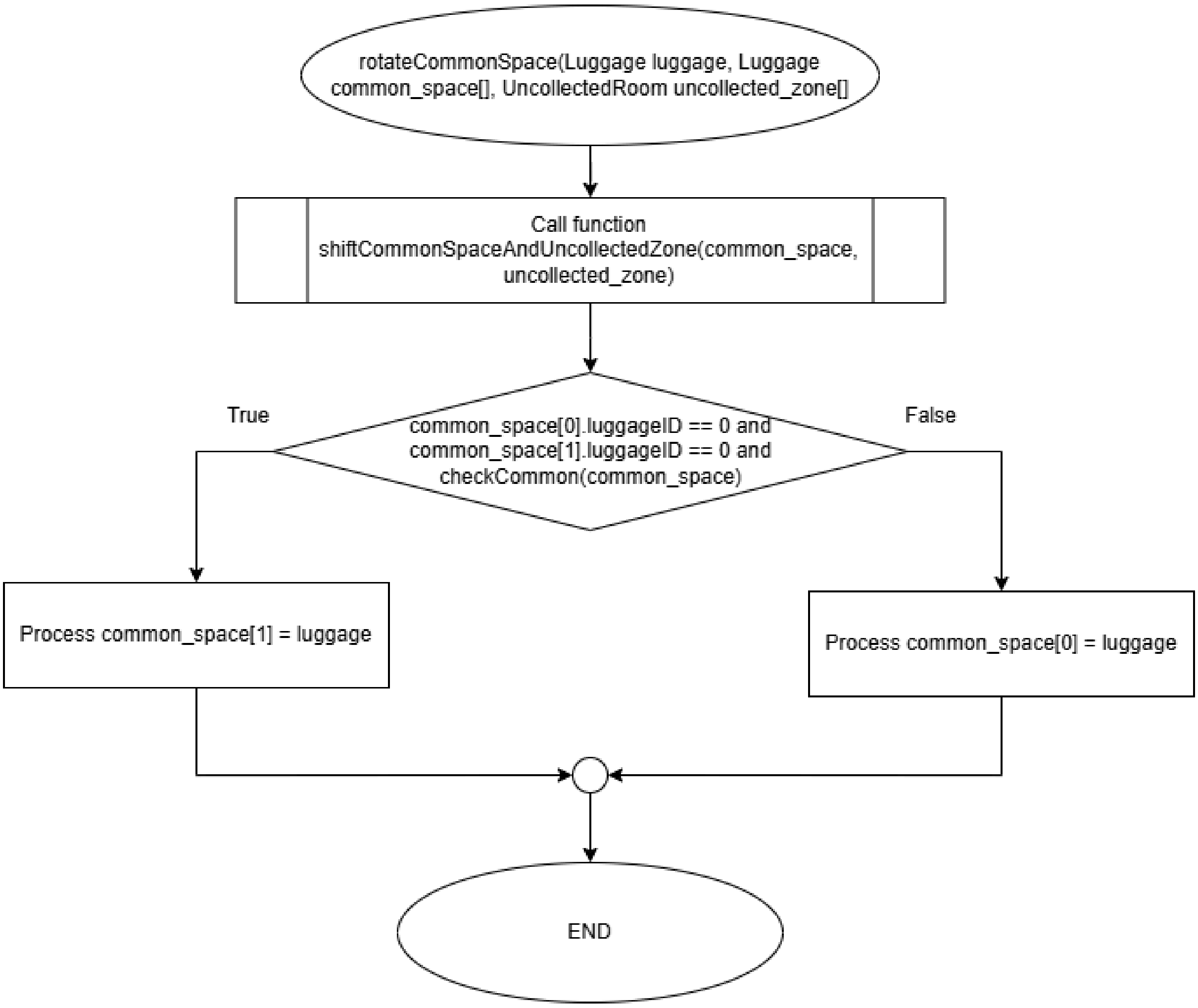

collectFromUncollectedZone(UncollectedRoom uncollected_zone[], int total_rooms)
Declare option
Declare luggageID, owner
Initialise foundRoom = 0, foundIndex = 0
input luggageID
Initialise i = 0
False
i < total_rooms
True
Initialise j = 0
False
j < uncollected_zone[i].inventory.size()
True
uncollected_zone[i].inventory[j].luggageID == luggageID
False
Process j++
True
Process foundRoom = i
Process foundIndex = j
Process i++
foundRoom != -1
False
True
foundRoom != -1
False
True
Display luggageID + "found in room " + uncollected_zone[foundRoom].room_id
Input owner
True
uncollected_zone[foundRoom.inventory[foundIndex].Owner == owner
False
Display "Identity verified!"
Display "Name verification failed."
Display luggageID + "not found"
uncollected_zone[foundRoom].removeLuggage(luggageID)
Display "successfully collected."
input option
option == 'Y' || option == 'y'
Display "Thank you for using the luggage collection service"
END

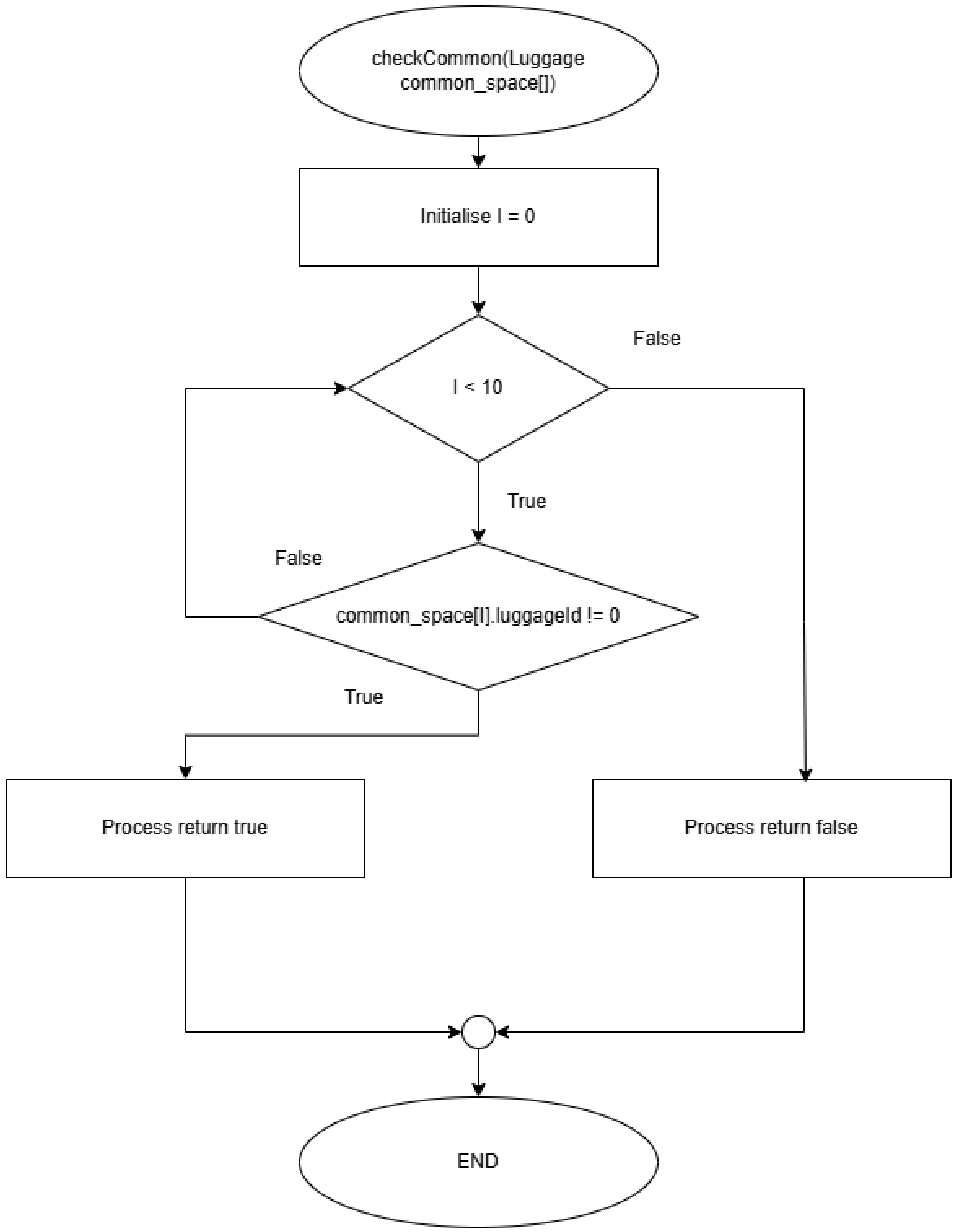


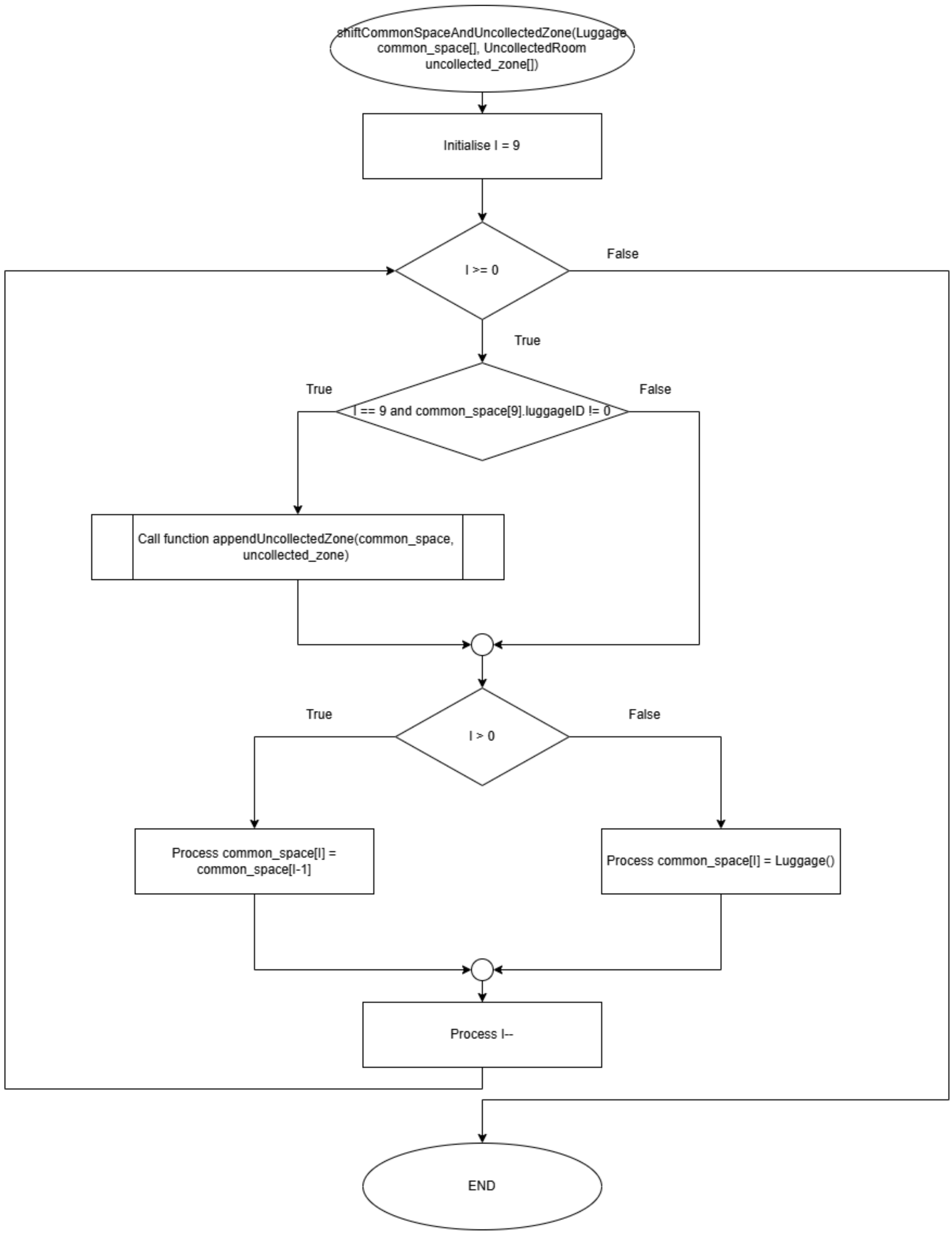
rotateLuggageCycle(queue<Luggage>& ready queue, Luggage common_space[], UncollectedRoom uncollected_zone[]
Initialize counter = 0
!ready_queue.empty() or checkCommon(common_space)
False
True
Call function displayLuggagePosition(ready_queue, common_space, uncollected_zone, counter)
Display 10 seconds timeframe to enter data
initialise LID = 0
LID = call function checkInput(10)
True
LID != 0
False
Call function collectLuggageAtCommonZone(common_space, LID)
True
!ready_queue.empty()
False
Call function rotateCommonSpace(ready_queue.front(), common_space, uncollected_zone)
Call function shiftCommonSpaceAndUncollectedZone(common_space, uncollected_zone)
Process ready_queue.pop()
counter += 10
Call function displayLuggagePosition(ready_queue, common_space, uncollected_zone, counter)
END

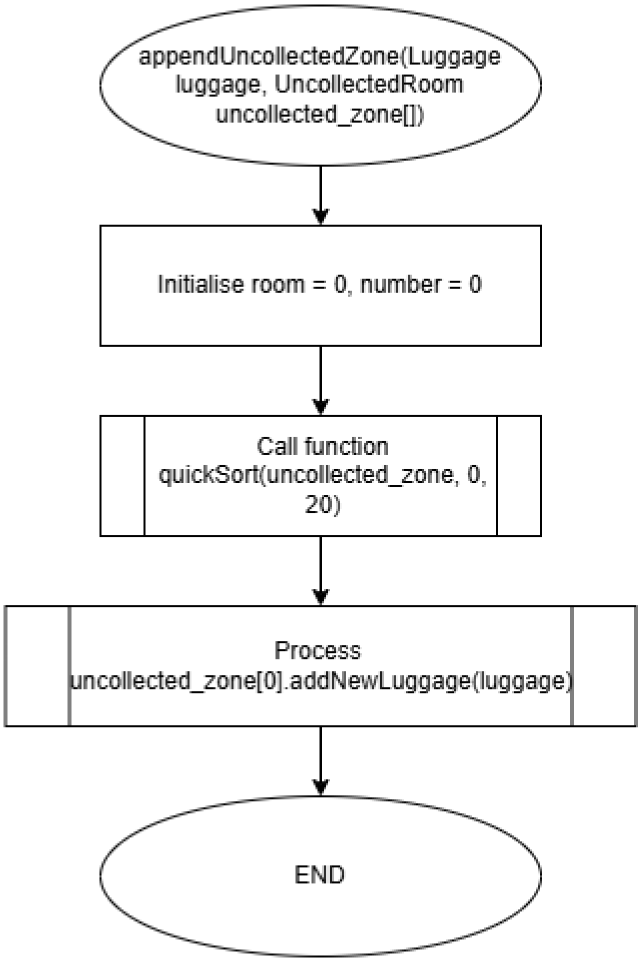

collectedLuggageAtCommonZone(Luggage common_space[], int id)
initialise id = 0
I < 10
False
True
Process I++
common_space[I].luggageID = id
False
True
Process common_space[j] = common_space[j-1]
Initialise j = i
j > 0
False
True
Process common_space[j] = common_space[j-1]
Process j--
Process common_space[0] = {}
Display luggage not found message
Process return 0
END

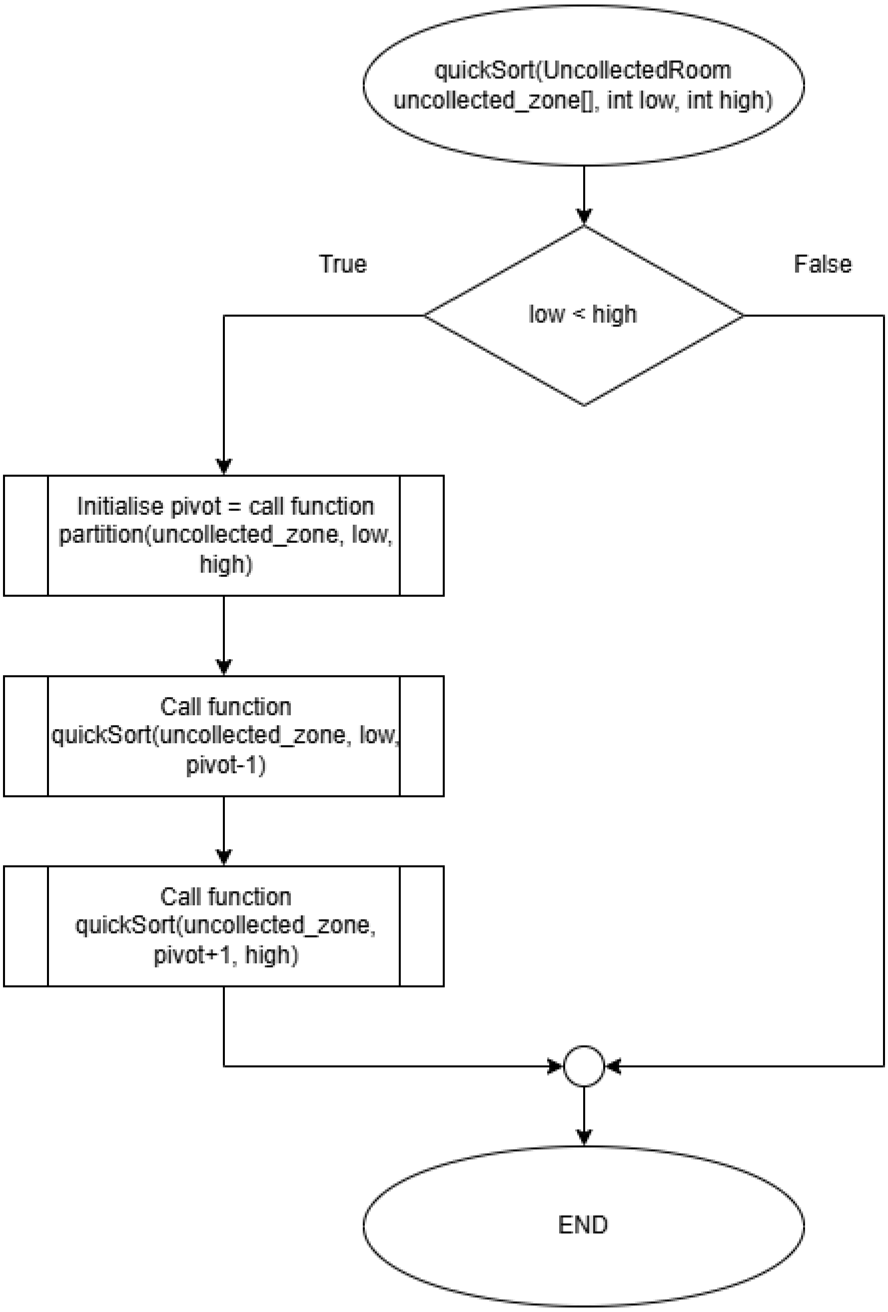

checkInput(int timeoutSeconds)
Initialise fd_set
Initialise timeval tv;
Call function FD_ZERO(&fds)
Call function FD_SET(STDIN_FILENO, &fds)
tv.tv_sec = timeoutSeconds
tv.tv_usec =0
True
function select(STDIN_FILENO + 1, &fds, NULL, NULL, &tv) > 0
False
Declare id
Input id
Process return id
Display timeout message
Process return 0
END

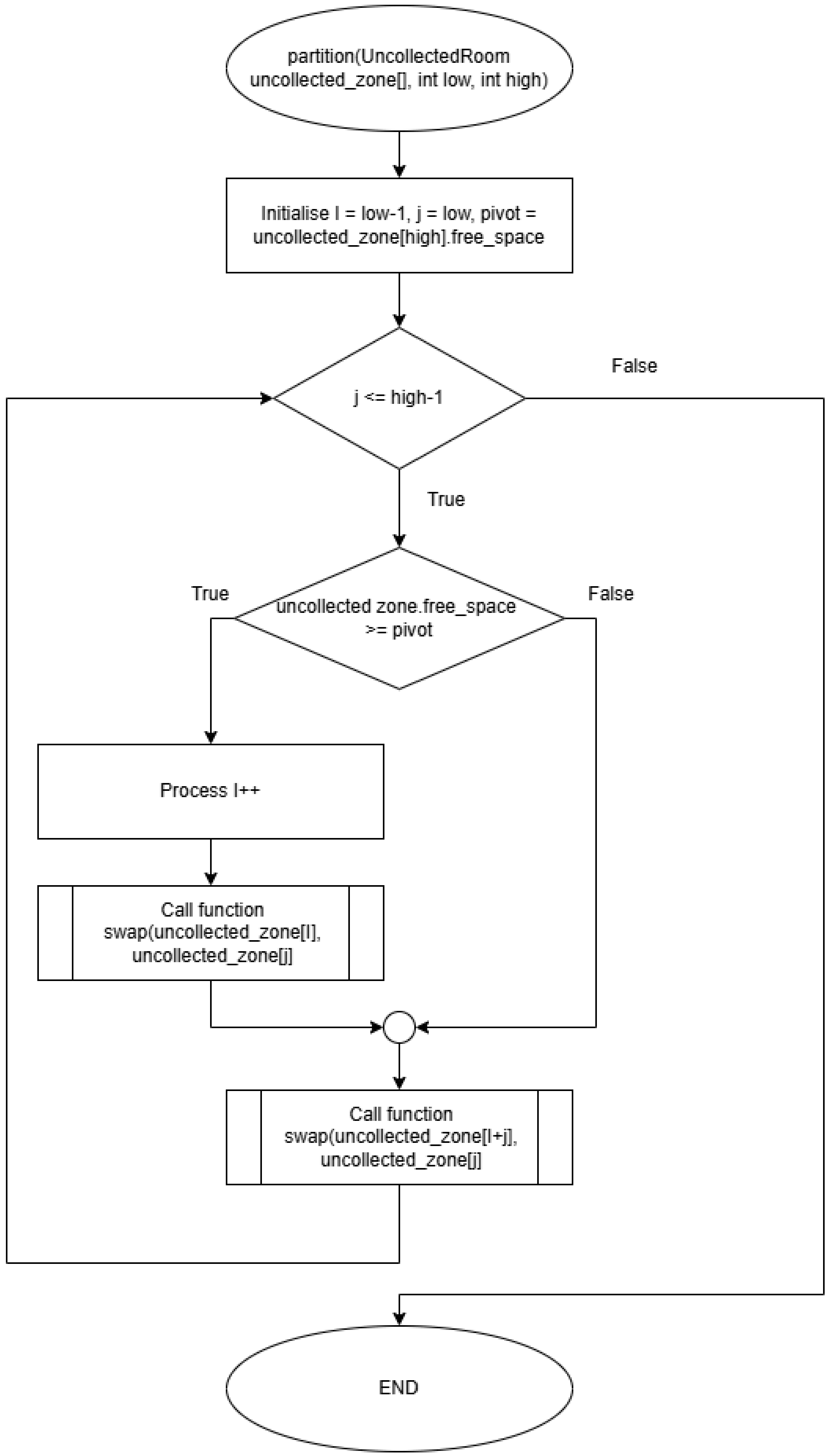

rotateCommonSpace(Luggage luggage, Luggage common_space[], UncollectedRoom uncollected_zone[]
Call function shiftCommonSpaceAndUncollectedZone(common_space, uncollected_zone)
True
common_space[0].luggageID == 0 and common_space[1].luggageID == 0 and checkCommon(common_space)
False
Process common_space[1] = luggage
Process common_space[0] = luggage
END


checkCommon(Luggage common_space[])
Initialise I = 0
I < 10
False
True
False
common_space[I].luggageId != 0
True
Process return true
Process return false
END

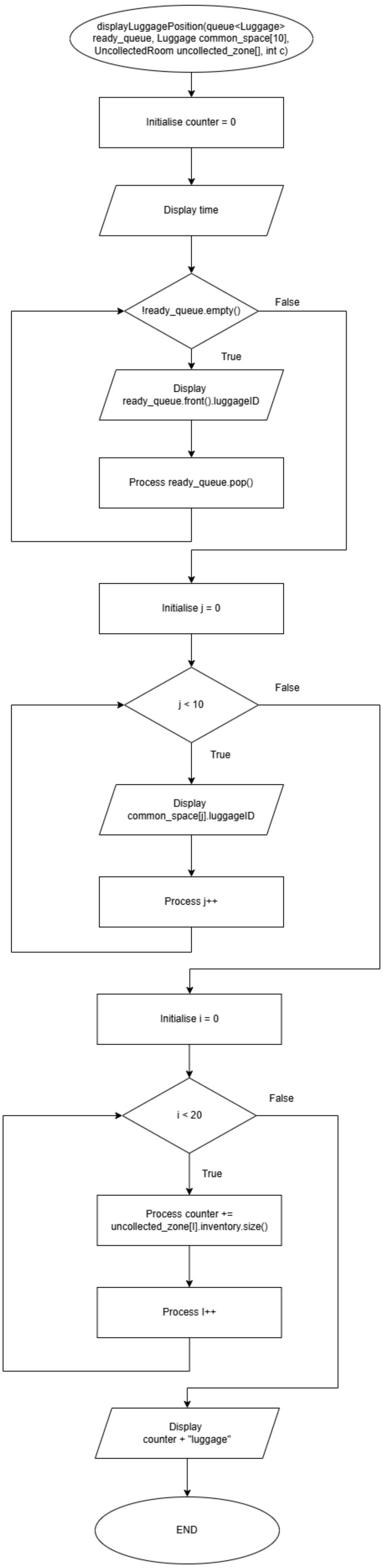

shiftCommonSpaceAndUncollectedZone(Luggage common_space[], UncollectedRoom uncollected_zone[])
Initialise I = 9
I >= 0
False
True
I == 9 and common_space[9].luggageID != 0
True
False
Call function appendUncollectedZone(common_space, uncollected_zone)
I > 0
True
False
Process common_space[I] = common_space[I-1]
Process common_space[I] = Luggage()
Process I--
END

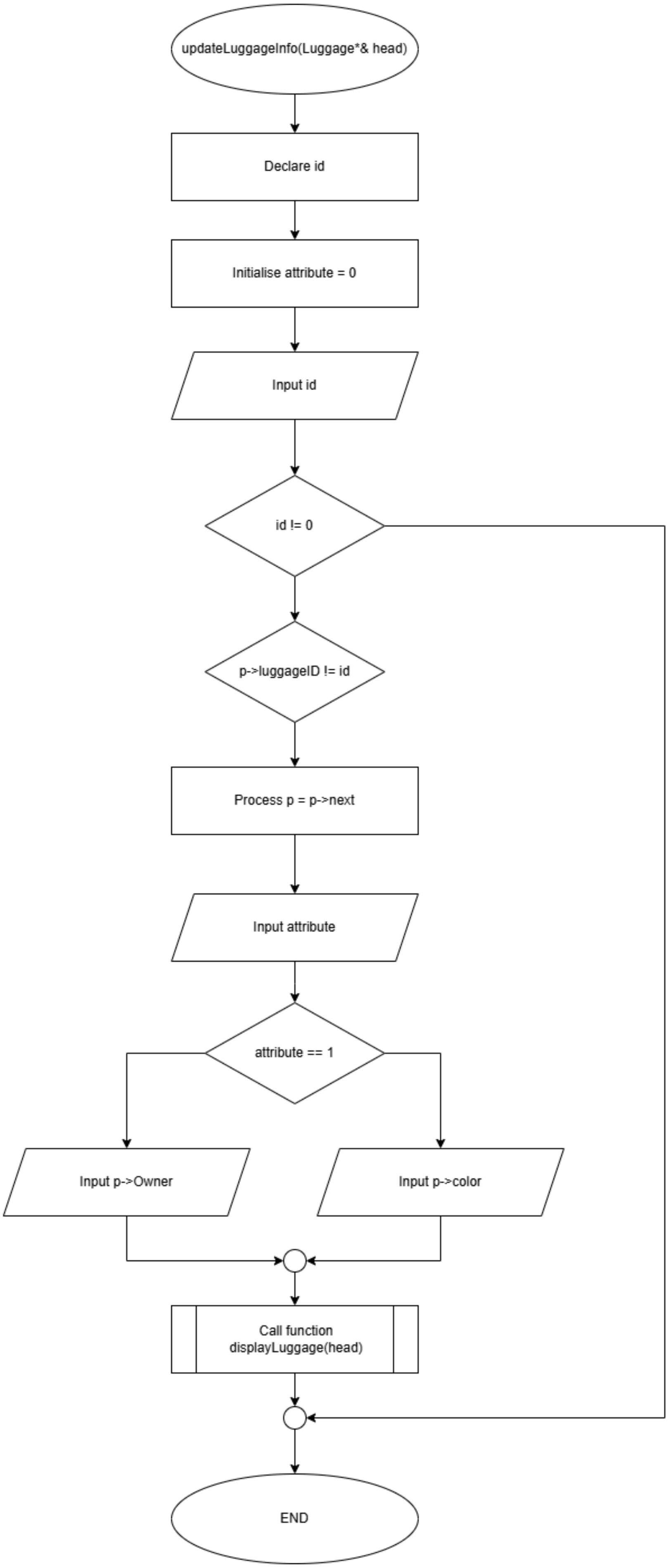

appendUncollectedZone(Luggage luggage, UncollectedRoom uncollected_zone[])
Initialise room = 0, number = 0
Call function quickSort(uncollected_zone, 0, 20)
Process uncollected_zone[0].addNewLuggage(luggage)
END

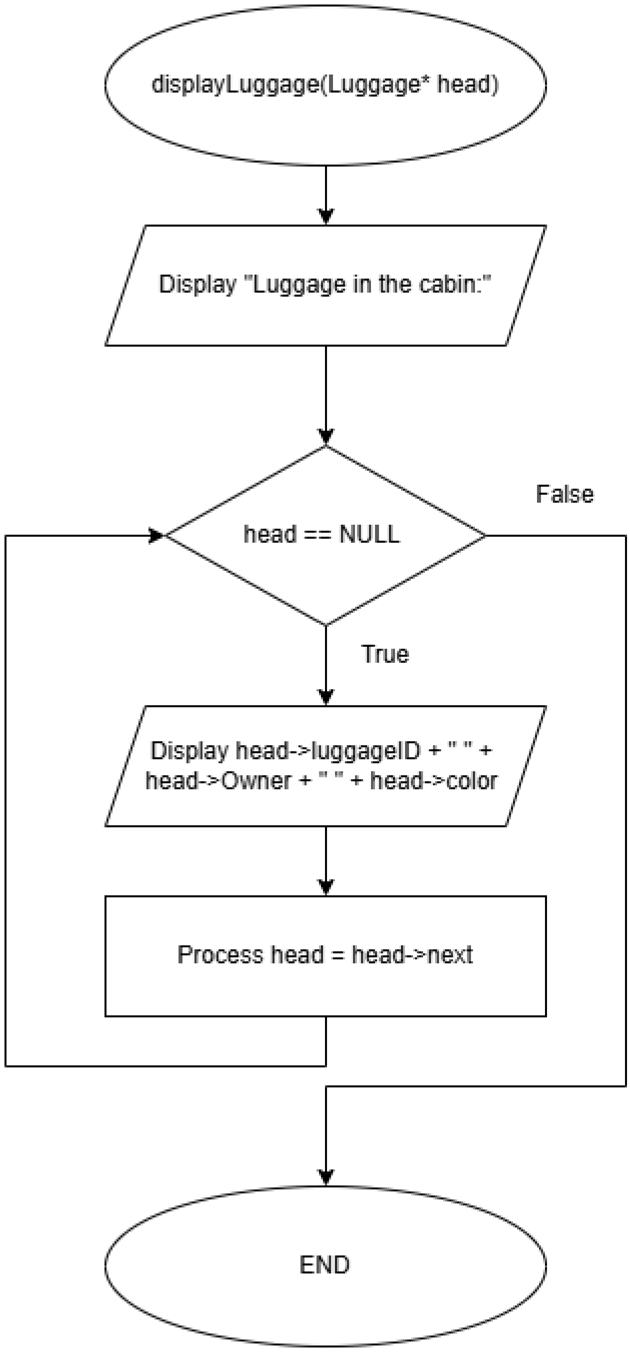

quickSort(UncollectedRoom uncollected_zone[], int low, int high)
True
low < high
False
Initialise pivot = call function partition(uncollected_zone, low, high)
Call function quickSort(uncollected_zone, low, pivot-1)
Call function quickSort(uncollected_zone, pivot+1, high)
END

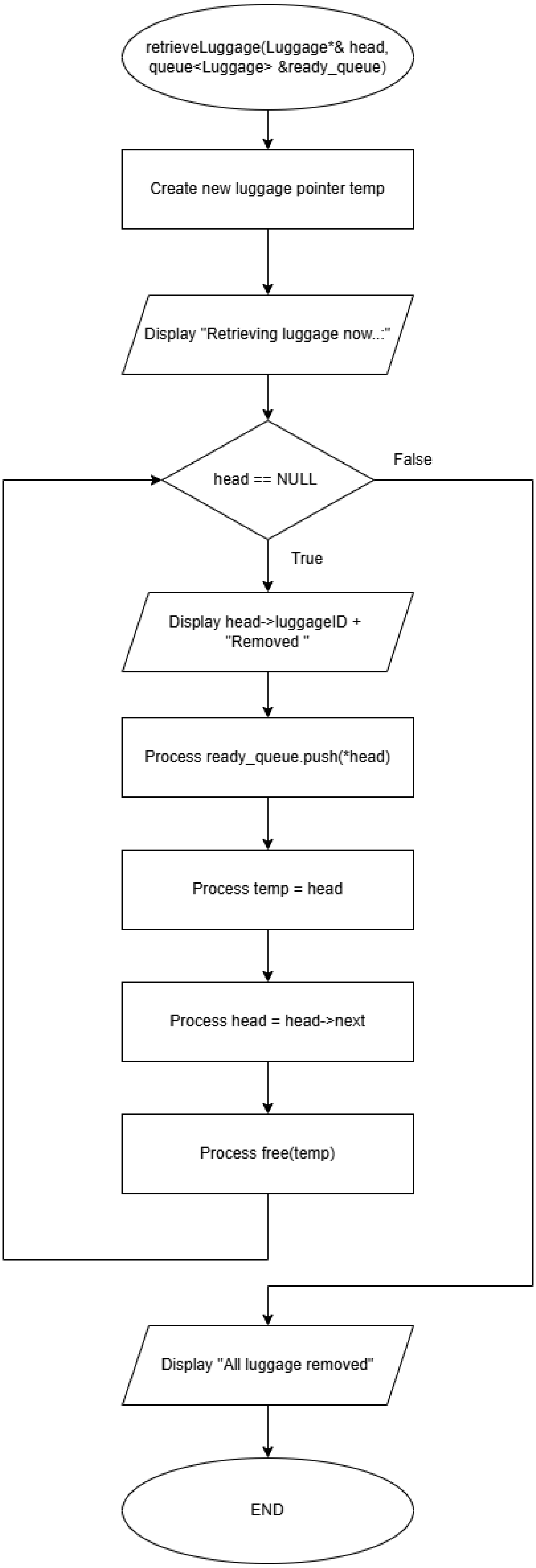


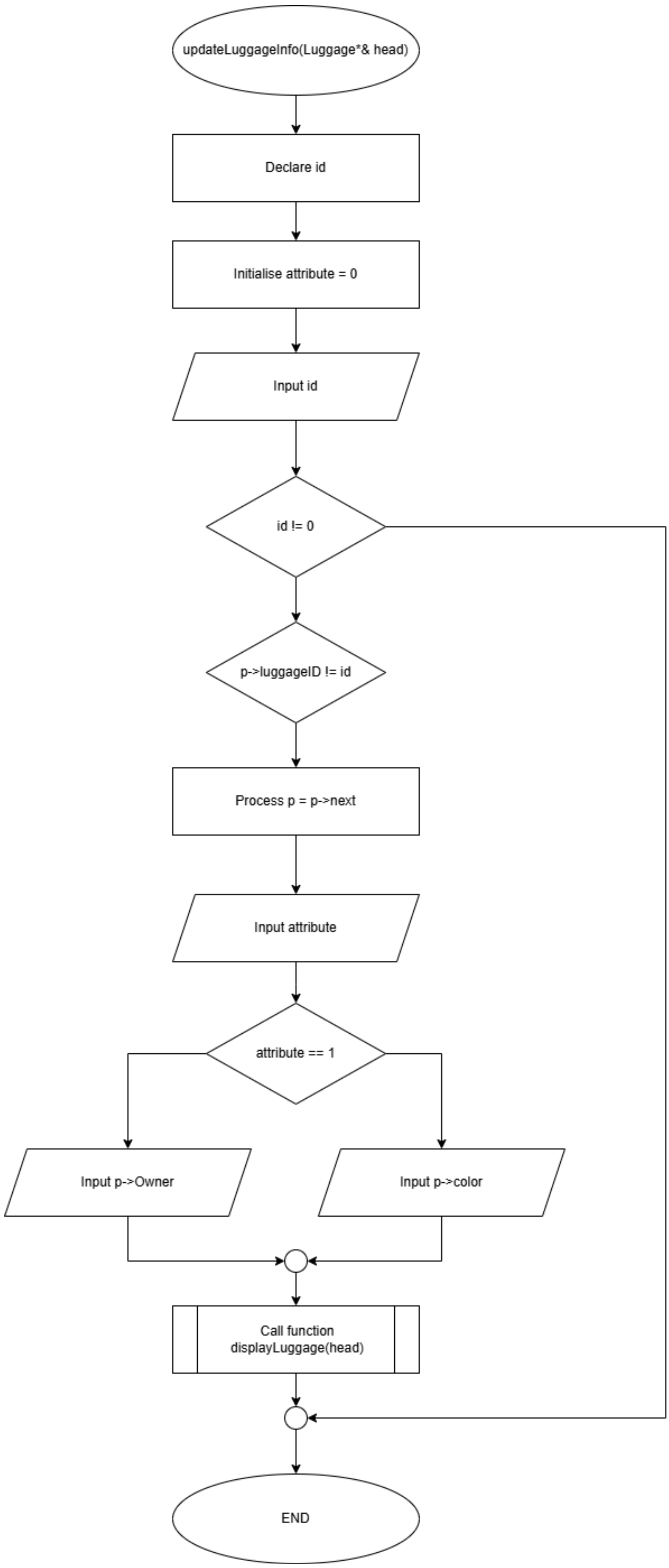
partition(UncollectedRoom uncollected_zone[], int low, int high)
Initialise I = low-1, j = low, pivot = uncollected_zone[high].free_space
j <= high-1
False
True
uncollected zone.free_space >= pivot
True
False
Process I++
Call function swap(uncollected_zone[I], uncollected_zone[j]
Call function swap(uncollected_zone[I+j], uncollected_zone[j]
END

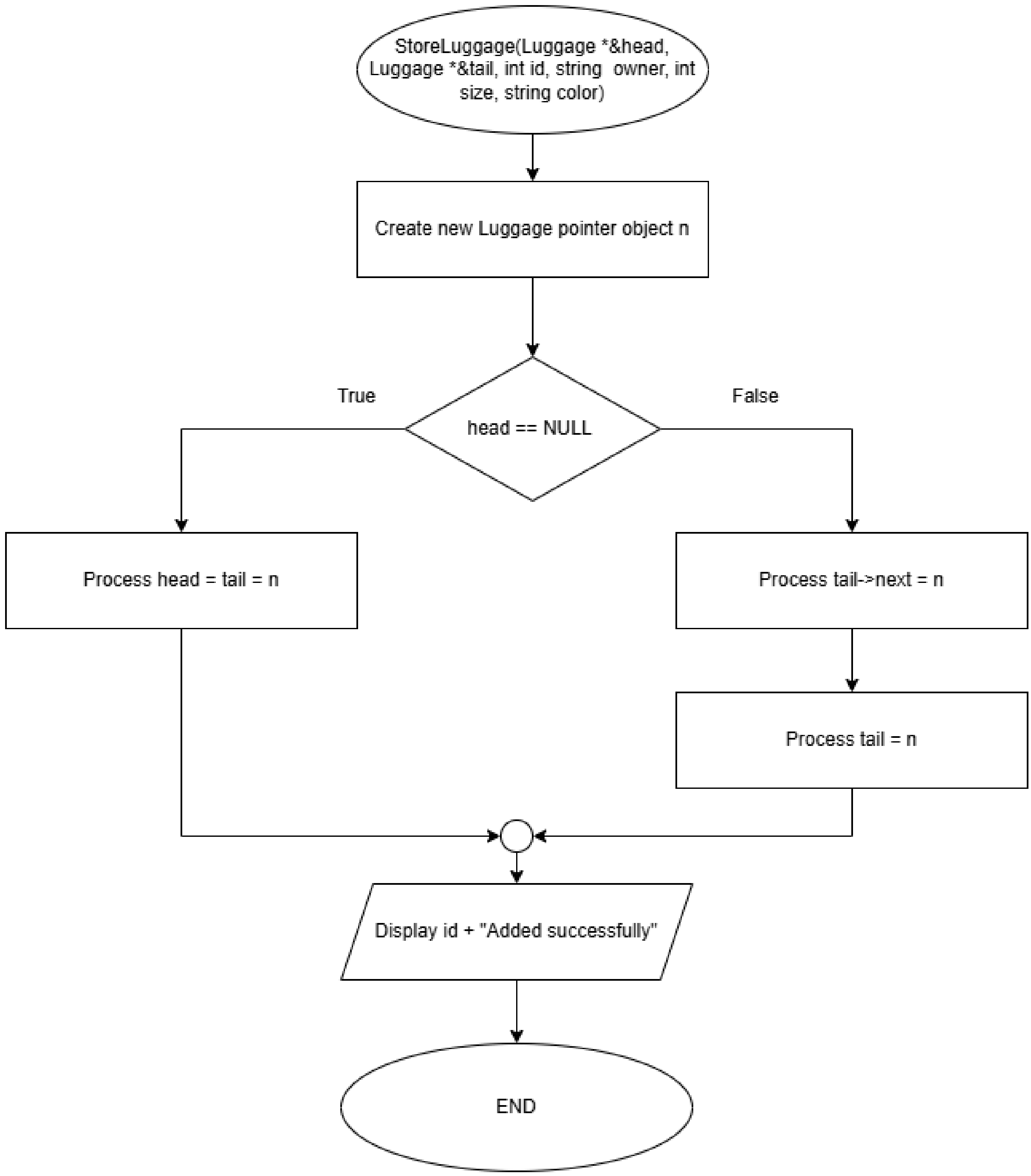

displayLuggagePosition(queue<Luggage> ready_queue, Luggage common_space[10], UncollectedRoom uncollected_zone[], int c)
Initialise counter = 0
Display time
!ready_queue.empty()
False
True
Display ready_queue.front().luggageID
Process ready_queue.pop()
Initialise j = 0
j < 10
False
True
Display common_space[j].luggageID
Process j++
Initialise i = 0
i < 20
False
True
Process counter += uncollected_zone[i].inventory.size()
Process I++
Display counter + "luggage"
END


updateLuggageInfo(Luggage*& head)
Declare id
Initialise attribute = 0
Input id
id != 0
p->luggageID != id
Process p = p->next
Input attribute
attribute == 1
Input p->Owner
Input p->color
Call function displayLuggage(head)
END


displayLuggage(Luggage* head)
Display "Luggage in the cabin:"
head == NULL
False
True
Display head->luggageID + " " +
head->Owner + " " + head->color
Process head = head->next
END


retrieveLuggage(Luggage*& head, queue<Luggage> &ready_queue)
Create new luggage pointer temp
Display "Retrieving luggage now..:"
head == NULL
False
True
Display head->luggageID + "Removed "
Process ready_queue.push(*head)
Process temp = head
Process head = head->next
Process free(temp)
Display "All luggage removed"
END


updateLuggageInfo(Luggage*& head)
Declare id
Initialise attribute = 0
Input id
id != 0
p->luggageID != id
Process p = p->next
Input attribute
attribute == 1
Input p->Owner
Input p->color
Call function
displayLuggage(head)
END


StoreLuggage(Luggage *&head, Luggage *&tail, int id, string owner, int size, string color)
Create new Luggage pointer object n
head == NULL
True
False
Process head = tail = n
Process tail->next = n
Process tail = n
Display id + "Added successfully"
END

5.0 Conclusion

In conclusion, a luggage management system that handles passengers' luggage effectively, verifies ownership, and keeps the retrieval process organized provides a much better experience for both passengers and airport staff. By focusing on the key objectives mentioned earlier, this system gives passengers confidence that their belongings are safe while also making airport operations run more smoothly. The simple layout of ready queues, common collection spaces, and designated uncollected zones makes it easy for passengers to quickly pick up their luggage, reducing waiting times and stress.

On the other hand, the accurate verification steps increase the system's reliability and transparency, preventing issues like lost or stolen luggage. This not only makes operations more efficient but also provides passengers with a greater sense of security.

Overall, the introduction of this comprehensive luggage management system shows a clear focus on both passenger convenience and smooth airport operations, making air travel more dependable, efficient, and enjoyable for everyone involved.

To sum up, this luggage management system will create a better experience for passengers and contribute to a positive environment for airport staff as well.